

Apache NiFi 2.x — Datenflüsse entwerfen, betreiben und skalieren

Architektur, Flow Design, Wiederverwendbarkeit, Performance und Deployment

Dr.-Ing. Grigory Devadze

2026-06-07

Agenda — Apache NiFi 2.x

Teil 1 — Verstehen und erste Flows bauen

- Einstieg, NiFi 2.x, Architektur
- Canvas, FlowFiles, Queues, Provenance
- Erster Flow, Expression Language, Parameter Contexts
- Übungen und Debrief

Teil 2 — Professionalisieren und betreiben

- Process Groups, Flow Definitions, Git-Versionierung
- Fehlerpfade, Retry, Quality, Wait/Notify
- Performance, Backpressure, Repositories, Cluster
- Security, Multi-User, TLS, Deployment, Migration

| Modul 1 — Einführung in Apache NiFi 2.x

Architektur, Datenflussmodell und Einordnung im Stack.

Was ist Apache NiFi?

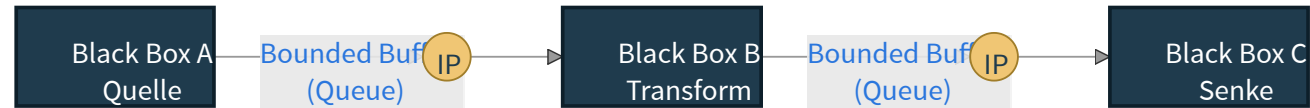
- **Flow-basierte Plattform** zur Orchestrierung, Steuerung und Beobachtung von Datenbewegung
- entwickelt für **Dateningest, Routing, Transformation und Systemintegration**
- stark, wenn Daten aus vielen Quellen in **kontrollierte, nachvollziehbare Pfade** überführt werden
- Fokus auf **visuelle Flow-Modellierung**, Betriebsnähe und schnelle Änderbarkeit

NiFi ist weniger eine Rechen-Engine und mehr eine **steuerbare Datenflussfabrik**.

Flow-Based Programming — die Idee hinter NiFi

NiFi ist eine **konkrete Umsetzung des Flow-Based-Programming-Paradigmas** (J. Paul Morrison, IBM, 1971). Wer die FBP-Begriffe kennt, liest NiFi-Architekturbilder anders.

Kernsatz der FBP-Definition: „Eine Anwendung ist ein Netz aus **Black-Box-Prozessen**, die über vordefinierte **Verbindungen Information Packets** austauschen — die Verbindungen sind **außerhalb** der Prozesse spezifiziert.“



FBP-Begriffe und ihre NiFi-Entsprechung

FBP-Begriff (Morrison)	NiFi-Begriff	Was es macht
Black Box / Process	Processor	die eigentliche Arbeit (Routing, Parsing, I/O)
Information Packet (IP)	FlowFile	das transportierte Datenpaket = Content + Attribute
Bounded Buffer	Connection / Queue	Puffer zwischen zwei Prozessoren, mit Priorität und Backpressure
Subnet	Process Group	gekapseltes Teilnetz, von außen wie eine Black Box behandelt
Scheduler	Flow Controller	vergibt Threads und Ressourcen, „das Hirn“ der Instanz

Konsequenz: Verbindungen liegen **außerhalb** der Prozessoren — deshalb lassen sich Black Boxes **endlos neu verdrahten**, ohne sie intern zu ändern. Genau das ist der Grund, warum Sie in NiFi Flows umbauen, ohne Code zu refactoren.

Das Kernmodell: Daten fließen als FlowFiles

- Ein **FlowFile** besteht aus **Content** + **Attributes**
- Prozessoren verändern entweder den **Inhalt**, die **Metadaten** oder den **Pfad**
- Verbindungen zwischen Prozessoren bilden **Queues**
- Scheduling, Prioritäten, Backpressure und Provenance machen den Flow **steuerbar und auditierbar**

Praktische Folge: In NiFi modelliert man nicht nur Verarbeitung, sondern auch **Transport, Kontrolle und Transparenz**.

Wofür wird NiFi typischerweise eingesetzt?

- **IoT / Edge** — Sensor- und Ereignisdaten sammeln, filtern, weiterleiten
- **ETL / ELT-Vorstufen** — Dateien, APIs, Datenbanken und Message-Systeme koppeln
- **Echtzeitanalysen** — Ereignisströme an Analytics-, Search- oder Streaming-Systeme anbinden
- **Systemintegration** — alte und neue Systeme ohne harte Punkt-zu-Punkt-Kopplung verbinden
- **Governance-sensitive Umgebungen** — wenn Herkunft, Steuerbarkeit und Fehlerpfade sichtbar sein müssen

NiFi vs. Kafka Streams vs. Apache Spark

Kriterium	NiFi	Kafka Streams	Spark / Structured Streaming
Primärer Fokus	Orchestrierung von Datenflüssen	Stream Processing in Code	Verteilte Datenverarbeitung
Interaktionsmodell	visuell, flow-basiert	Java-Code, library-first	Code + Cluster-Engine
Stärken	Routing, Integration, Provenance, Ops	event-getriebene Logik nahe Kafka	große Datenmengen, komplexe Transformationen
Typische Grenze	keine High-End-Compute-Engine	weniger visuell, weniger UI-gesteuert	schwergewichtig für Integrationsklebstoff
Ideal, wenn	viele Systeme gekoppelt werden	Kafka das Zentrum ist	analytische Rechenlast dominiert

Faustregel: NiFi gewinnt dort, wo **sichtbare Datenflüsse, Integrationsbreite und operative Kontrolle** wichtiger sind als reine Rechenleistung.

NiFi vs. Airflow — nicht verwechseln

Frage	NiFi	Airflow
Denkt in	Datenströmen und FlowFiles	Tasks, DAGs und Zeitplänen
Stark bei	data-in-motion, Routing, Protokollvielfalt	batch-orchestrierten Jobs und Python-zentrischen Pipelines
Typischer Stil	visuelles Operieren im laufenden Fluss	geplante Workflow-Schritte mit Scheduler
Ergänzt wen?	NiFi kann Daten bewegen, vorbereiten, puffern	Airflow kann nachgelagerte Jobs koordinieren

Synergie: Airflow triggert NiFi-Pipelines per REST-API; NiFi liefert Daten in von Airflow orchestrierte Stages.

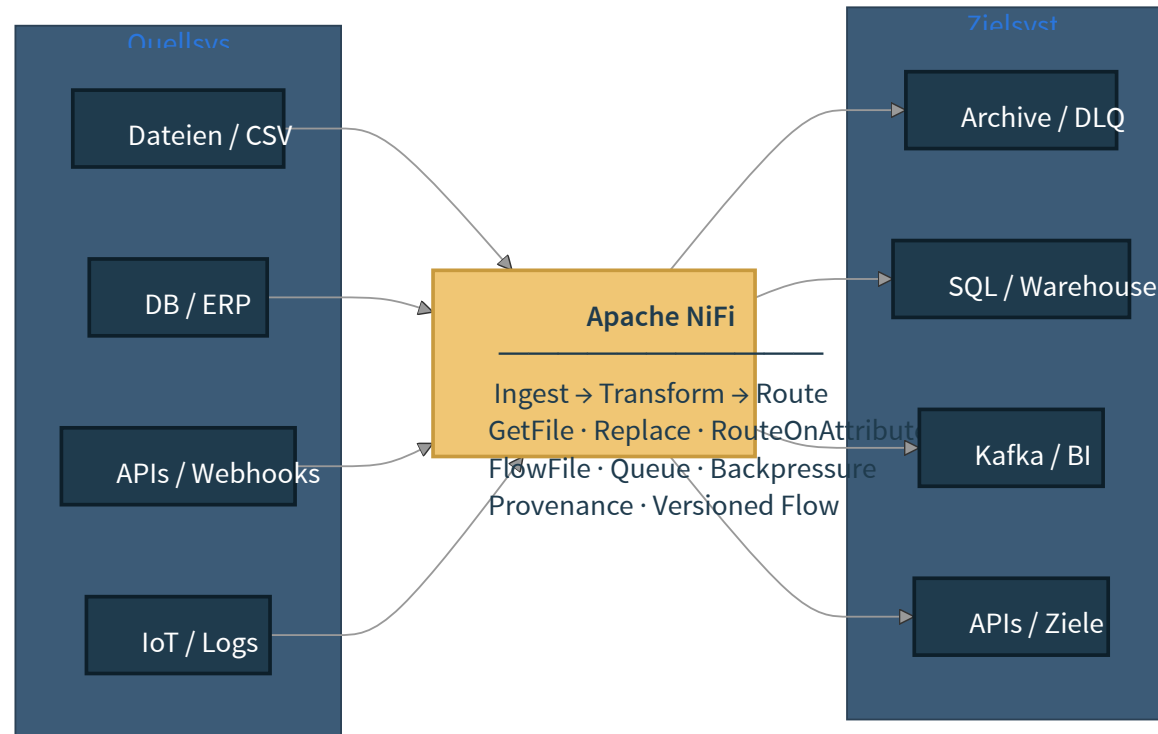
Entscheidungskriterien — wann welches Werkzeug?

- **NiFi:** viele Quell-/Zielsysteme, heterogene Protokolle, Routing- und Provenance-Anforderungen, operativ steuerbare Datenbewegung
- **Airflow:** Python-Logik, komplexe Job-Abhängigkeiten, Zeitplan-getriebene Batches
- **Kafka Streams:** Stream-Processing in Java/Scala, Kafka als Datenrückgrat, kontinuierliche Transformationen
- **Spark / Structured Streaming:** verteilte Compute-Last, Joins über große Datenmengen, SQL-zentrierte Pipelines

Wann NiFi nicht ideal ist:

- aggressive Inline-Transformationen mit Window-Aggregationen → Kafka Streams oder Spark
- ML-Inferenz mit hoher Last → spezialisierte Serving-Stacks
- pure Code-First-Teams ohne Toleranz für visuelle Modellierung → Airflow + DBT

Ein durchgängiger Beispiel-Flow



Demo — Minimaler Live-Flow für den Einstieg

Konkreter Live-Pfad:

1. `GenerateFlowFile` erzeugt eine Testnachricht
2. `UpdateAttribute` ergänzt `source=demo` , `priority=high` , `target=landing`
3. `ReplaceText` formatiert den Payload als sauberen Datensatz
4. `PutFile` schreibt das Ergebnis in einen Zielordner
5. Provenance öffnen und genau **ein** FlowFile von Start bis Ziel verfolgen

Modul 2 — Grundlagen und Benutzeroberfläche

Mit dem Canvas arbeiten und einen ersten Flow bauen.

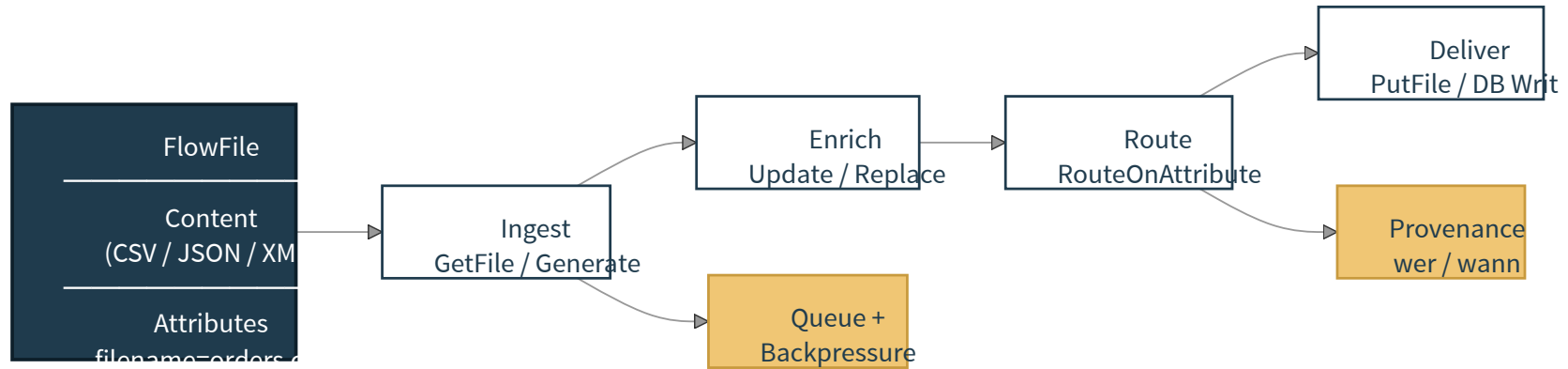
Die wichtigsten Elemente der NiFi-Oberfläche

- **Canvas / Arbeitsfläche** — hier werden Flows visuell zusammengesetzt
- **Prozessoren** — ausführbare Bausteine für Lesen, Transformieren, Schreiben, Routen
- **Connections / Queues** — Übergänge und Puffer zwischen Verarbeitungsschritten
- **Process Groups** — Container für Struktur, Wiederverwendung und Abstraktion
- **Controller Services** — zentrale Infrastruktur-Bausteine (DB-Pools, Schema-Reader, SSL)
- **Flow Definitions / Versioned Flows** — exportier- und versionierbare Process Groups

Einen einfachen Flow Schritt für Schritt bauen

1. Quelle hinzufügen, z. B. `GenerateFlowFile` oder `GetFile`
2. Verbindung mit Queue anlegen
3. Transformationsschritt ergänzen, z. B. `ReplaceText`
4. Zielprozessor konfigurieren, z. B. `PutFile`
5. Beziehungen (`success` , `failure` , `unmatched`) bewusst verdrahten
6. Komponenten validieren, starten und Verhalten beobachten

FlowFile-Struktur verstehen



Queues, Prioritäten und Backpressure

- Queues puffern Daten zwischen Verarbeitungsschritten
- Prioritäten steuern, **welche** FlowFiles zuerst weiterverarbeitet werden
- Backpressure begrenzt **Objektanzahl** und **Datenvolumen** in einer Queue
- So verhindert NiFi, dass langsame Downstream-Systeme unkontrolliert überflutet werden

Praxisfrage: Nicht nur „Wie schnell läuft mein Flow?“, sondern auch „Wie verhält er sich unter Last und Störung?“.

Data Provenance: Warum NiFi so gut nachvollziehbar ist

- Provenance zeigt, **wann** ein FlowFile erzeugt, gelesen, verändert, geroutet oder gelöscht wurde
- hilfreich für **Fehlersuche**, Audits, Compliance und Ursachenanalyse
- erlaubt die Rekonstruktion von Pfaden durch komplexe Flows
- schafft Transparenz zwischen Entwicklung, Betrieb und Fachseite

Leitidee: Gute Integrationsplattformen bewegen Daten nicht nur — sie machen die Bewegung **sichtbar**.

FlowFile Replay — Debugging mit einem Klick

Provenance zeigt den Pfad — **Replay** fährt ihn neu ab. Möglich, weil FlowFile-Content und -Attribute in den Repositories **immutable** vorgehalten werden: jeder Verarbeitungsschritt erzeugt einen neuen Zustand, der alte bleibt referenzierbar.

- in der **Provenance-Ansicht** ein Event auswählen (z. B. `ATTRIBUTES_MODIFIED` oder `CONTENT_MODIFIED`)
- *Show Lineage* öffnet den vollständigen Verlauf des FlowFiles
- „**Replay**“-Button an jedem Knoten — die exakte Bytes/Attribute des damaligen Zustands erscheinen wieder in der **Upstream-Queue** des nachfolgenden Prozessors
- ideal zum Reproduzieren von Fehlern: gleicher Inhalt, gleicher Attribut-Stand, aktuelle Logik

Drei typische Anwendungsfälle:

1. „*Hat unser Validator das wirklich abgelehnt?*“ — Replay vor `ValidateRecord` mit aktuell geändertem Schema
2. „*Warum ist die Anreicherung leer?*“ — Replay vor `LookupRecord` , Stammdaten-Cache prüfen
3. „*Patch in Produktion eingespielt — wirkt er rückwirkend?*“ — Replay derselben FlowFiles gegen den neuen Prozessor



Stolperfalle: Replay funktioniert nur, solange der **Content Claim** im `content_repository/` noch existiert. Aggressive Retention/Archive-Settings können dem Replay den Boden entziehen — Retention bewusst auf das Debugging-Fenster auslegen.

Übung 1 — Ziel & Setup

Ziel: Einen Mini-Flow bauen, der Daten erzeugt, verändert und lokal ablegt — und einen FlowFile vollständig in der Provenance verfolgen.

Setup:

- NiFi 2.x lokal (Docker oder Standalone)
- Zielordner `/tmp/nifi-uebung1/` schreibbar
- Single-User-Login verfügbar

Übung 1 — Schritte

1. `GenerateFlowFile` erzeugt Testdaten (Custom Text, alle 30 sec)
2. `ReplaceText` ersetzt Platzhalter im Inhalt
3. `UpdateAttribute` ergänzt `source=uebung1`, `priority=normal`
4. `PutFile` schreibt das Ergebnis in `/tmp/nifi-uebung1/`
5. Provenance öffnen und den Weg eines FlowFiles nachvollziehen

Übung 1 — Akzeptanzkriterien & Debrief

Erwartetes Ergebnis: mindestens 5 Dateien in `/tmp/nifi-uebung1/` mit verändertem Inhalt und gesetzten Attributen.

Akzeptanzkriterien:

- Dateien sind tatsächlich verändert (Platzhalter ersetzt)
- Attribute sind gesetzt (`source` , `priority`)
- Provenance zeigt für ein FlowFile mindestens 4 Events (CREATE, ATTRIBUTES_MODIFIED, CONTENT_MODIFIED, SEND)

Debrief-Fragen: Unterschied Inhalt vs. Attribute? Welche Infos in der Queue sichtbar? Welche Beziehungen wären zusätzlich zu verdrahten?

Erweiterung (optional): `failure` -Beziehung von `PutFile` auf eine zweite `PutFile` -Senke in `quarantine/` legen.

| Modul 3 — Flow Design & Expression Language

Attribute, Standardprozessoren, Parameter Contexts und dynamisches Routing.

Standardprozessoren für die Datenaufbereitung

- `SplitText` — große Textdateien in handhabbare Teile zerlegen
- `ReplaceText` — Inhalte standardisieren, Muster ersetzen, Formate glätten
- `UpdateAttribute` — Routing-Metadaten, Klassifikationen, technische Marker ergänzen
- `RouteOnAttribute` — FlowFiles anhand von Regeln auf verschiedene Pfade verteilen

Didaktik: Diese vier Prozessoren genügen, um viele Grundmuster professioneller Flows zu erklären.

Die innere Struktur eines FlowFiles gezielt nutzen

- Attribute werden zur **Steuerlogik** des Flows
- Konventionen für Attributnamen erhöhen Wartbarkeit und Teamfähigkeit
- saubere Trennung von **Payload**, **technischer Metadaten** und **fachlicher Klassifikation** reduziert Chaos
- je mehr ein Flow wächst, desto wichtiger werden **Benennung, Kapselung und Standardisierung**

NiFi Expression Language — das Grundprinzip

- Ausdruckssprache für dynamische Eigenschaften und Regeln
- arbeitet typischerweise auf **FlowFile-Attributen**
- erlaubt String-Operationen, Bedingungen, Vergleiche, Datumsfunktionen und Verkettungen
- häufige Einsatzfelder: Dateinamen, Routing, Zielpfade, API-Parameter, Logging-Kontexte
- in der offiziellen NiFi-Dokumentation beschrieben

- **`${filename:toLowerCase():replace(' ', '_')}`** — Dateinamen standardisieren
- **`${priority:equals('high')}`** — High-Priority-Flow erkennen
- **`${now():format("yyyy/MM/dd")}`** — Datum in Zielpfade schreiben

Record-orientierte Verarbeitung und API-Anbindung

- **Record-Prozessoren als Kern:** `QueryRecord`, `ValidateRecord`, `PutDatabaseRecord`
- aus Text-/Datei-Routing wird **schema- und datensatzbewusste** Verarbeitung
- `InvokeHTTP` sowie `HandleHttpRequest` / `HandleHttpResponse` öffnen API-Integrationen
- REST-API und CLI helfen, Flows **automatisiert auszurollen und zu administrieren**

Praxiswert: Produktive Teams gehen über das Canvas hinaus — hin zu Automatisierung und recordsicherer Verarbeitung.

Szenario 1 — Demo: Record-Flow für ETL und Qualitätsprüfung

Konkreter Aufbau:

1. `GetFile` liest eingehende CSV- oder JSON-Dateien
2. `ValidateRecord` prüft Schema und Pflichtfelder
3. `QueryRecord` filtert oder projiziert fachlich relevante Datensätze
4. `PutDatabaseRecord` schreibt den sauberen Pfad in die Zieldatenbank
5. ungültige Datensätze gehen über `PutFile` in Quarantäne

Didaktischer Mehrwert: Aus einem Demo-Flow wird ein echter **ETL-/Qualitätssicherungs-Flow**.

Szenario 2 — REST/API-Ingest mit Retry und Rate-Limit

Anforderung: stündlich Datensätze von einer externen REST-API holen, in DB schreiben.

1. **CRON-getriggert:** `GenerateFlowFile` mit Cron `0 0 * * * ?` als Initiator
2. `InvokeHTTP` ruft die API auf (Authentifizierung über sensitive Parameter)
3. **Erfolg (2xx)** → `ConvertRecord` → `PutDatabaseRecord`
4. **5xx** → `RetryFlowFile` (Backoff über Penalty Duration) — nach ausgeschöpften Retries eskalieren
5. **429** → `RetryFlowFile` mit Backoff oder `ControlRate` ; Quarantäne erst nach definiertem Max-Retry
6. **4xx** → Quarantäne-Pfad in `quarantine/4xx/`

Stolperfalle: Ohne `RetryFlowFile` -Limit dreht sich der Flow bei Dauer-5xx ewig. Maximum Retry Count immer setzen.

Szenario 3 — Multi-Source-Fan-in

Anforderung: drei Quellen (Datei-Drop, REST, Kafka-Topic) müssen vor dem DB-Write zusammenfließen.

- **Variante A: Wait/Notify** — drei `Notify` (je Quelle, gleicher `Release Signal Identifier = ${batch.date}`), ein `Wait` mit Counter 3 vor `PutDatabaseRecord`
- **Variante B (einfacher): MergeContent** — ein gemeinsames Input Funnel + `MergeContent` mit `Minimum Number of Entries = 3` und kurzer `Max Bin Age`
- **Entscheidungskriterium:** Quellen unabhängig + nur „mindestens N“ → `MergeContent`. Jede Quelle eindeutig identifiziert → `Wait/Notify`.

API- und Betriebsautomatisierung

- `InvokeHTTP` zeigt, wie NiFi externe REST-Dienste aufrufen kann
- `HandleHttpRequest` / `HandleHttpResponse` demonstrieren eingehende API-getriebene Flows
- die **REST-API** ist relevant für automatisiertes Prüfen, Deployen und Administrieren
- die **NiFi-CLI** verbindet GUI-orientierte Entwicklung mit standardisiertem Betrieb

Typische Einsatzmuster der Expression Language

- **Dateinamen standardisieren** — Leerzeichen ersetzen, Suffixe ergänzen
- **Routing definieren** — hoch / niedrig priorisierte Datensätze trennen
- **Zielpfade dynamisieren** — Datum, Mandant oder Quellsystem in Ordnerpfade einbauen
- **Technische Marker setzen** — Fehlerklasse, Retry-Zähler, Umgebungsname
- **Properties kontextabhängig machen** — URLs, Topics oder Tabellen dynamisch ansprechen

Parameter Contexts und Controller Services

- **Parameter Contexts** ersetzen die alte Variable Registry und entkoppeln Flows von harten Werten
- **Parameter** werden in Properties als `#{paramName}` referenziert
- **Inheritance**: ein Parameter Context kann von einem anderen erben — typisch *base* → *dev* / *test* / *prod*
- **Controller Services** bündeln wiederkehrende Infrastrukturkonfigurationen (DB-Pools, SSL-Kontexte, Schema-Registries)
- Zusammengekommen entstehen **portierbare und umgebungsfähige** Flows

Architekturgedanke: Ein professioneller Flow ist nicht nur funktional, sondern auch **portierbar und wartbar** — und kennt **keine** Klartext-Secrets.

Sensitive Parameters und Umgebungen

- **Sensitive Parameters** werden in der UI nicht angezeigt und intern verschlüsselt
- **Sensitive Parameter Providers** binden externe Secret Stores an: HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager, Kubernetes Secret
- pro Umgebung **eigener Parameter Context** mit gleichen Parameter-Namen, anderen Werten
- Migrations-Tipp aus 1.x: alte **Variable-Registry-Einträge** als Parameter in einen neuen Context übernehmen



Stolperfalle: Klartext-Secrets in Prozessor-Properties landen auch im Versioned Flow im Git. Immer als sensitive Parameter plus Provider modellieren.

LookupRecord — Stammdaten-Anreicherung im Record-Stream

Anreicherung ist der häufigste „Join“ in NiFi — und passiert idealerweise **innerhalb eines Record-Flows**, nicht als getrennter DB-Roundtrip pro FlowFile.

- **LookupRecord** liest jedes Record im FlowFile, schlägt ein Feld in einem Controller Service nach und schreibt das Ergebnis in ein Zielfeld
- **DatabaseRecordLookupService** — Lookup gegen eine relationale Tabelle (z. B. Kunden-Stammdaten)
- **SimpleKeyValueLookupService** / **SimpleCsvFileLookupService** — schneller In-Memory-Lookup für kleine Mappings
- **DistributedMapCacheLookupService** — geteilter Cache (z. B. Geräte-IDs → Standort) im Cluster

Praxisregel: Stammdaten unter ~100 k Einträgen → CSV/Map-Cache. Größere oder häufig wechselnde Daten → Database-Lookup mit Cache-TTL.

Externe Skripte einbinden — ExecuteScript & ExecuteStreamCommand

Wenn weder Standardprozessor noch Expression Language reicht — bevor Sie einen **eigenen NAR-Build** anfangen, die beiden Skript-Wege prüfen:

- **ExecuteScript** — Inline-Skript in **Groovy**, **Jython**, **Clojure** oder ECMAScript. Zugriff auf FlowFile-Session, Attribute, Content
- **InvokeScriptedProcessor** — wie **ExecuteScript**, aber als wiederverwendbarer Prozessor mit eigenen Properties
- **ExecuteStreamCommand** — ruft eine **externe Binary** auf, pipet den FlowFile-Content rein/raus (Python-Script, **jq**, **xmllint**, ML-Inferenz-CLI)

Wann was? Kleine Logik im Flow → **ExecuteScript**. Wiederverwendbarer Spezial-Prozessor → **InvokeScriptedProcessor**. Vorhandenes externes Tool → **ExecuteStreamCommand**.

 **Stolperfalle:** Skript-Prozessoren laufen pro FlowFile — die JVM bezahlt jedes Mal Initialisierungskosten. Für Hot Paths Caches und statische Felder im Skript nutzen, sonst NAR-Build planen.

Kafka-Anbindung — Consume / Publish / Record-Varianten

Kafka-Prozessoren sind versioniert — die `_2_6`-Familie ist in NiFi 2.x der Standard, ältere `_0_10` / `_1_0` / `_2_0`-Varianten sind Legacy.

Prozessor	Zweck
<code>ConsumeKafka_2_6</code>	Topic abonnieren, jede Message als FlowFile
<code>PublishKafka_2_6</code>	FlowFile-Content als Message in ein Topic schreiben
<code>ConsumeKafkaRecord_2_6</code>	mehrere Messages pro FlowFile als Record-Batch — viel performanter
<code>PublishKafkaRecord_2_6</code>	Record-FlowFile als mehrere Messages publizieren

Faustregel: Strukturierte Daten (JSON/Avro) → immer die **Record-Variante** verwenden. Spart pro FlowFile-Overhead und ermöglicht `QueryRecord` / `ValidateRecord` im selben Pfad.

Sicherheit: SASL/SSL über die Kafka-Client-Properties (`security.protocol` , `sasl.mechanism` , `ssl.truststore.location`) — die gehören in eine **Parameter Group**, nicht in jeden Prozessor einzeln.

Cloud- und Format-Connectoren — Landschaft

NiFi bringt **ab Werk** Prozessoren für die großen Cloud-Stacks und gängige Dateiformate mit — kein Connector-Marktplatz nötig.

Bereich	Prozessoren (Auswahl)
AWS	PutS3Object / FetchS3Object / ListS3 , PutKinesisStream / ConsumeKinesisStream , PutDynamoDB / GetDynamoDB , PutSQS / GetSQS
Azure	PutAzureBlobStorage_v12 / FetchAzureBlobStorage_v12 , ConsumeAzureEventHub , PutAzureCosmosDBRecord , PutAzureDataLakeStorage
GCP	PutGCSObject / FetchGCSObject , PublishGCPubSub / ConsumeGCPubSub , PutBigQuery
NoSQL	PutMongoRecord / GetMongo , PutCassandraRecord , PutElasticsearchRecord
Formate	ConvertExcelToCSVProcessor , ConvertRecord (Parquet ↔ Avro ↔ JSON ↔ CSV), ParseFixedWidth via Schema-Registry

Cloud-Connectoren — Format-Muster und Auth

Muster für Format-Konvertierung: `ListenHTTP` → `ConvertExcelToCSVProcessor` → `ConvertRecord` (CSV → Parquet) → `PutS3Object` — drei Prozessoren, kein Custom-Code, ein Schema im **Avro Schema Registry**.

 **Wahl-Faustregel:** Cloud-native Service-Account-Auth (IAM Role / Managed Identity) immer dem Klartext-Secret vorziehen — die Credential-Provider-Services dafür stehen unter *Controller Services* → *CredentialsProviderService*.

MergeContent — Strategien

MergeContent sammelt mehrere kleine FlowFiles zu einem großen Bündel — kritisch für Downstream-Effizienz (S3-PUTs, DB-Batches, Kafka-Throughput).

Zwei Merge-Strategien:

Strategy	Wann verwenden
Bin-Packing	„N FlowFiles oder X Bytes sammeln, dann mergen“ — Throughput-Optimierung
Defragment	FlowFiles wurden vorher durch <code>SplitText</code> / <code>SplitJson</code> zerlegt — Original wiederherstellen (über <code>fragment.identifier</code>)

Record-Variante: **MergeRecord** macht dasselbe für **Record-FlowFiles** (CSV/JSON/Avro), nicht für rohen Content — bevorzugt, wenn der Bündel-Output später wieder als Records weiterverarbeitet wird.

MergeContent — Sizing-Properties

Alle Properties wirken gemeinsam — die erste erfüllte Bedingung schließt den Bin:

- `Minimum Number of Entries` / `Maximum Number of Entries` — Bin-Grenzen an Stückzahl
- `Minimum Group Size` / `Maximum Group Size` — Bin-Grenzen an Bytes
- `Max Bin Age` — **Timeout**: Bin wird auch unvollständig geschlossen (verhindert „Bin füllt sich nie“)
- `Maximum Number of Bins` — gleichzeitig offene Bins (für Correlation-Attribute, z. B. pro Tenant ein Bin)
- `Correlation Attribute Name` — gruppiert FlowFiles, die denselben Attributwert teilen, in denselben Bin

Faustregel: S3/Parquet → Bins ab **10–100 MB**. Kafka → Bins ab **1 000–10 000 Messages**. Immer `Max Bin Age` setzen, sonst hängen FlowFiles bei niedrigem Volumen ewig.

NiFi als HTTP-Server — synchrone APIs anbieten

Der Standard-Inbound (`ListenHTTP` , `HandleHttpRequest` / `HandleHttpResponse`) macht aus NiFi einen **vollwertigen REST-Server** — inklusive synchroner Antworten an den Aufrufer.



- **HandleHttpRequest** öffnet den Port, nimmt Requests entgegen, korreliert sie mit dem späteren Response-Prozessor über `http.context.identifier`
- **HandleHttpResponse** schreibt den finalen Status zurück — **muss** für jeden Pfad (success + failure) verdrahtet sein, sonst hängt der Aufrufer im Timeout
- **StandardHttpContextMap** — Controller Service, der die offenen Requests cached (Default: 100 gleichzeitig, 30 s Timeout)

Typische Einsatzfälle:

- **Synchrone Anreicherung** (Frontend ruft NiFi, NiFi schlägt nach, antwortet sofort)
- **Webhook-Empfang** (GitHub, Slack, Stripe, SAP-Cloud-Integration)
- **Mini Dashboard** (HTML-Renderer aus einem Record-Flow, kein zusätzlicher Webserver nötig)

Übung 2 — Ziel & Setup

Ziel: Datensätze je nach Priorität oder Quellsystem auf unterschiedliche Pfade verteilen — und die Routing-Regeln über Parameter steuerbar machen.

Setup:

- Übung 1 abgeschlossen
- Parameter Context `uebung2` mit Parametern `target_high`, `target_normal` angelegt
- Zielordner schreibbar

Übung 2 — Schritte

1. `GenerateFlowFile` oder Testdatei mit Attributen vorbereiten
2. `UpdateAttribute` setzt `priority=high|normal` und `source=erp|crm`
3. `RouteOnAttribute` mit Regeln `${priority:equals('high')}` und Catch-All `unmatched`
4. je Pfad `PutFile` in unterschiedliche Ordner (Parameter-gesteuert via `#{target_high}` / `#{target_normal}`)
5. Regeln dokumentieren, Testfälle pro Priorität durchspielen

Übung 2 — Akzeptanzkriterien & Debrief

Erwartetes Ergebnis: Files landen je nach Priorität in unterschiedlichen Ordnern.

Akzeptanzkriterien:

- High-Priority-Files in `#{target_high}` , Normal in `#{target_normal}`
- Wechsel des Parameter-Werts ändert das Ziel ohne Re-Deploy des Flows
- `unmatched` -Pfad ist explizit verdrahtet, nicht ignoriert

Debrief-Fragen: Wo machen Parameter den Unterschied? Was passiert ohne Catch-All? Wie sähe das mit `QueryRecord` aus?

Erweiterung (optional): Sensitive Parameter für ein simuliertes API-Token referenzieren, im Versioned Flow committen, Klartext-Abwesenheit im Git verifizieren.

Modul 4 — Anwendungsentwicklung und Wiederverwendbarkeit

Modulare Flows, Fehlerpfade und Teamarbeit.

Process Groups, Flow Definitions, Versioned Flows

- **Process Groups** strukturieren große Flows in fachliche oder technische Domänen
- wiederkehrende Muster lassen sich kapseln: Ingest, Validierung, Standardisierung, Auslieferung
- **Flow Definitions** (JSON-Export einer Process Group) ersetzen die alten XML-Templates aus 1.x
- **Versioned Flows** = Process Groups unter Versionskontrolle eines **Flow Registry Clients**
- gute Modularisierung reduziert visuelle Komplexität und senkt Fehlerrisiko

Kernaussage: Templates sind in NiFi 2.x kein Thema mehr — wer sie nennt, hat 1.x-Material in der Hand.

Wiederverwendbare Flow-Architektur

Typisches Muster:

1. Eingangsgruppe pro Quellsystem
2. gemeinsame Normalisierungsschicht
3. Routing- und Qualitätsregeln
4. Zielsystem-spezifische Ausgabepfade
5. Querschnittsdienste für Logging, Parameter, Credentials

Zielbild: Nicht jeder Flow ist ein Unikat — gute Teams bauen **Bibliotheken aus Bausteinen**.

Fehlerbehandlung professionell modellieren

- Beziehungen wie `failure`, `retry`, `unmatched` nie ignorieren
- **Dead-Letter-Queues** trennen dauerhaft problematische Daten vom Normalfluss
- **Retry-Pfade** helfen bei temporären Fehlern, z. B. Netzwerk- oder API-Ausfällen
- dynamisches Routing erlaubt differenzierte Reaktionen je nach Fehlerklasse

Faustregel: Ein produktiver Flow hat immer einen Plan für **Happy Path**, **Transient Failure** und **Hard Failure**.

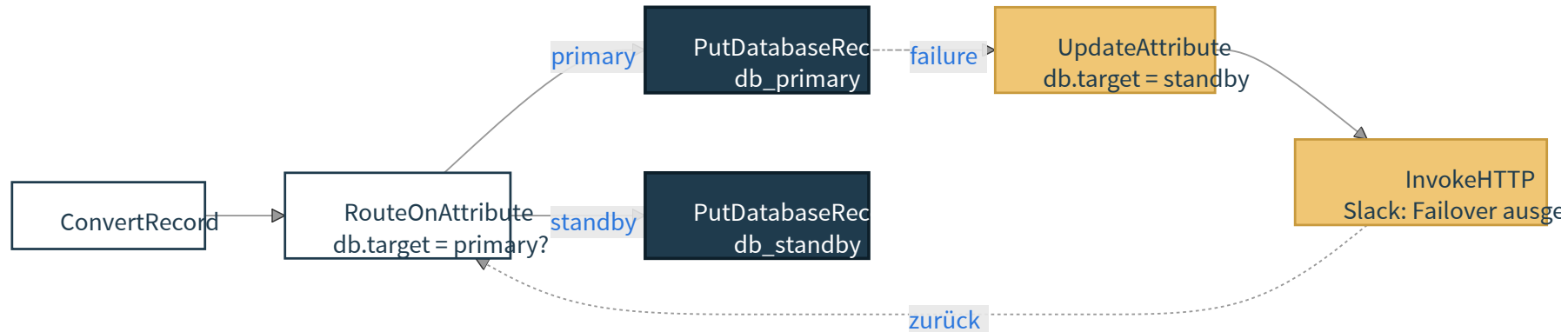
Database-Failover automatisieren

Wenn der primäre DB-Endpoint ausfällt, soll der Flow **selbst** auf die Standby-Instanz wechseln — ohne Eingriff um 3 Uhr nachts.

Bausteine:

- Zwei **DBCPConnectionPool** -Services — **db_primary** und **db_standby** (gleiches Schema, andere Hosts)
- ein **UpdateAttribute** setzt **db.target=primary** als Default
- die DB-Schreib-Prozessoren (**PutDatabaseRecord** / **ExecuteSQL**) referenzieren den Pool dynamisch via **\${db.target}** -gesteuertes Routing über zwei parallele Zweige
- der **failure** -Pfad triggert **UpdateAttribute** → **db.target=standby** und führt das FlowFile per Funnel wieder in den Schreibzweig zurück
- ein **MonitorActivity** auf **db_primary.success** erkennt, wenn die Primary wieder gesund ist → Reset auf **primary**

Database-Failover — Flow-Skizze



⚠ **Stolperfalle:** Endlos-Loop, wenn beide DBs unten sind. Ein **retry**-Zähler-Attribut (**UpdateAttribute** inkrementiert **failover.retries**, **RouteOnAttribute** bricht ab **> N**) gehört dazu — sonst legt der Flow den Cluster lahm.

Flow Registry Clients in NiFi 2.x — Git als Primärweg

- NiFi 2.x bringt **Flow Registry Clients** als integrierte API mit
- **Empfohlen:** Git-basierte Clients gegen **GitHub / GitLab / Bitbucket / Azure DevOps**
- Versionierung wie für Code: **Branches, Pull Requests, Code Review, Tags/Releases, Rollback**
- pro Process Group ein **Bucket/Repository** + Pfad innerhalb des Repos
- Rollback durch Auswahl einer früheren Version im Flow-Registry-Client-Dialog

Scheduling: Jobs zeitgesteuert ausführen

Jeder Prozessor hat einen **Scheduling Strategy**-Tab. In NiFi 2.x sind zwei Modi der Standard:

- **Timer driven** — fester Abstand, z. B. `30 sec`, `5 min`. Default für die meisten Quellen.
- **CRON driven** — Quartz-Cron-Ausdruck. Standard für „werktags 06:00“, „jede volle Stunde“.

Beispiel-CRON: `0 0 6 ? * MON-FRI` — werktags 06:00 Uhr.

Concurrent Tasks steuert zusätzlich, wie viele Threads ein Prozessor parallel nutzt.

Scheduling-Muster für typische Anforderungen

- **Polling-Quelle** (`GetFile` , `ListSFTP` , `QueryDatabaseTable`) — Timer driven mit moderatem Intervall
- **Tagesendverarbeitung** — CRON-Trigger auf einer steuernden `GenerateFlowFile` , die den Flow startet
- **Bursts** — Timer driven mit kurzem Run Schedule, passenden Concurrent Tasks, Backpressure und Downstream-Schutz
- **Pausefenster** — CRON/Run Schedule, parameter-gesteuerte Route oder `ControlRate` bewusst modellieren; Prozessorstatus nicht als Release-Schalter missbrauchen

Jobabhängigkeiten: Wait und Notify

NiFi kennt keine zentralen DAG-Trigger wie Airflow. Abhängigkeiten werden über das **Wait/Notify-Pattern** modelliert:

- **Notify** setzt einen **Signal-Counter** in einem **Map-Cache-Controller-Service** (MapCacheServer + MapCacheClientService)
- **Wait** hält FlowFiles solange zurück, bis der erwartete Counter steht
- gemeinsamer Schlüssel ist eine **Release Signal Identifier**-Property — meist ein Attribut wie `${batch.id}` oder `${date}`

Typisches Muster: „Ziel-DB erst befüllen, wenn alle drei Quelldateien des Tages eingelaufen sind.“ — drei **Notify**, ein **Wait** (Counter 3) vor **PutDatabaseRecord**.

Wait/Notify — was praktisch zu wissen ist

- der **MapCacheServer** (separater Controller Service) muss laufen — sonst stille Fehler
- **Wait** hat ein **Expiration Duration** — gegen liegen gebliebene Signale
- Counter werden **nicht automatisch** zurückgesetzt — bewusst über **release** oder Reset-Flow leeren
- für **lineare Ketten** reicht oft eine simple Verbindung; Wait/Notify ist für **fan-in** über getrennte Flows

Alternative: Eine umschließende Process Group mit **MergeContent** (Mindestanzahl FlowFiles) löst viele „Sammele N Eingänge“-Fälle einfacher.

Demo — Release-Pfad mit Git-basiertem Flow Registry Client

Demo-Ablauf:

1. bestehende Process Group kapseln
2. Git-basierten Flow Registry Client auswählen (Repository, Branch, Pfad)
3. erste Version committen
4. Änderung am Fehlerpfad oder an einem Parameter durchführen
5. zweite Version committen
6. Diff und Versionshistorie im Flow-Dialog erklären
7. Rollback auf die vorherige Version live durchführen

Lernpunkt: Versionierung ist Teil professioneller Flow-Entwicklung — wie für Code, mit Branches, PRs und Rollback.

CI/CD für NiFi — Werkzeuge im Zusammenspiel

Git-Versionierung allein ist Versionierung — **Deployment-Automatisierung** braucht eine Pipeline. NiFi 2.x bringt die Bausteine mit, die ein Jenkins (oder GitHub Actions / GitLab CI) orchestrieren kann.

Komponente	Rolle
NiFi Registry / Git Registry Client	Source-of-Truth für Flow-Versionen
NiFi CLI / Toolkit (<code>nifi.sh</code> , <code>cli.sh</code>)	scriptbar — Import, Export, Promote, Parameter-Set
REST API	alles, was die CLI macht, ist auch direkt POSTbar
Jenkins / GitHub Actions	bindet beides zu einer Pipeline mit Stages

CI/CD für NiFi — Pipeline-Skizze

```
1 stage('Lint Flow JSON')      { sh 'nifi-cli.sh registry list-flows ...' }
2 stage('Promote DEV → TEST')  { sh 'nifi-cli.sh registry import-on-nifi --baseUrl $TEST_URL --flow $FLOW_ID'
3 stage('Smoke Test')          { sh 'curl -fsS $TEST_URL/api/health && pytest tests/' }
4 stage('Promote TEST → PROD') { input 'Promote to Production?'
5                               sh 'nifi-cli.sh registry import-on-nifi --baseUrl $PROD_URL --flow $FLOW_ID' }
```

Wert: Promotion zwischen DEV → TEST → PROD wird vom **Audit-fähigen Build** statt vom manuellen Klick gefahren. Sensitive Parameter pro Umgebung kommen aus dem Jenkins-Credential-Store oder dem Sensitive-Parameter-Provider — **nicht** aus dem Flow.

Übung 3 — Ziel & Setup

Ziel: Den Übungsflow aus 1+2 zu einem produktionsreifen, versionierten und rollback-fähigen Stand bringen.

Setup:

- Übungen 1+2 abgeschlossen
- lokaler Git-Server (Gitea) oder Test-Repo in GitHub/GitLab erreichbar
- Git-basierter Flow Registry Client in NiFi 2.x konfiguriert

Übung 3 — Schritte

1. Flow in eine **Process Group** „uebung3“ kapseln
2. **Parameter Context** anlegen (Pfade, Quellen, Ziel), inklusive einem sensitive Parameter
3. **Fehlerpfade** explizit modellieren — `failure`, `retry`, `unmatched` jeweils auf einen Quarantäne-Subflow
4. **Versioned Flow** anlegen — Process Group an den Git-basierten Flow Registry Client committen
5. eine bewusste **Breaking Change** einbauen (z. B. falscher Parameter-Wert), neue Version committen
6. **Rollback** auf die vorherige Version durchführen, Verhalten prüfen

Übung 3 — Akzeptanzkriterien & Debrief

Erwartetes Ergebnis: Process Group existiert in Git mit mindestens zwei Versionen, Rollback erfolgreich.

Akzeptanzkriterien:

- mindestens 2 Versionen im Git-Repository
- Rollback wieder aktiv, neue Version weiterhin im Git verfügbar
- keine Klartext-Secrets im Versioned Flow erkennbar
- alle Fehlerbeziehungen verdrahtet

Debrief-Fragen: Was wurde durch Modularisierung besser? Wie unterscheidet sich der Git-Rollback von einem manuellen Flow-Reset?

Erweiterung (optional): Pull-Request-Workflow im Git üben — Branch + PR + Review + Merge → erneuter Pull in NiFi.

Modul 5 — Performance, Cluster, Repositories

Lastverhalten verstehen und für Skalierung vorbereiten.

Welche Stellschrauben bestimmen die Performance?

- **Threading / Concurrent Tasks** je Prozessor
- **Run Schedule** und Trigger-Frequenz
- **Queue-Größen und Backpressure**
- **JVM-/Speicherzuweisung** und Repository-Verhalten
- Verhalten externer Zielsysteme: DB, Filesystem, API, Broker

Wichtig: Performanceprobleme entstehen in NiFi oft an den **Übergängen** — nicht nur im Prozessor selbst.

Thread-Pools, Speicher und Backpressure richtig lesen

- mehr Parallelität ist nur dann sinnvoll, wenn Downstream-Systeme mithalten können
- Backpressure schützt Stabilität, ist aber kein Ersatz für Architekturentscheidungen
- große Queues können Störungen abfedern, aber auch Symptome verstecken
- Speicher- und Repository-Konfigurationen beeinflussen Durchsatz, Latenz und Recovery-Verhalten

Clustering: Warum und wann?

- Cluster verteilen Last auf mehrere Knoten
- erhöhen Ausfallsicherheit und horizontale Skalierbarkeit
- verlangen saubere Sicht auf **State, Scheduling und Datenlokalität**
- sinnvoll, wenn Volumen, Verfügbarkeit oder Team-/Betriebsanforderungen steigen

Nicht jede Installation braucht ein Cluster — aber jede produktive Installation sollte die Frage bewusst beantworten.

Multinode-Architektur im Überblick

Thema	Worum es geht
Knotenkoordination	gemeinsamer Clusterzustand und Steuerung
Datenverteilung	welche Knoten welche Arbeit sehen oder übernehmen
Lastverteilung	gleichmäßige Nutzung von CPU, IO und Zielsystemen
Failover / Robustheit	Verhalten bei Knotenverlust oder Neustart
Betriebsmodell	Monitoring, Upgrades, Rollout-Fenster, Security

Site-to-Site (S2S) — NiFi-zu-NiFi-Verbindung

Wenn nicht alles in einem Cluster laufen kann oder soll: **Site-to-Site** ist NiFis eingebautes Protokoll zur Föderation zwischen Instanzen.

- **Edge → Core:** schlanke NiFi-Instanz am Standort sammelt Daten, leitet sicher an das zentrale Cluster weiter
- **DMZ-Bridge:** getrennte Instanzen für unsichere und sichere Netzzonen, S2S als kontrollierter Übergang
- **Cluster-übergreifende Lastverteilung:** zwei Cluster, einer schickt explizit an Remote Process Group des anderen
- **Reporting:** `SiteToSiteStatusReportingTask` pusht Metriken vom Produktiv- ins Monitoring-Cluster

Mechanik: Im Sender-Flow eine **Remote Process Group** anlegen, URL des Ziel-Clusters eintragen — die Ziel-Input-Ports erscheinen automatisch (sofern berechtigt).



Transport-Wahl: in NiFi 2.x ist **HTTP über TLS** der Default und einzige empfohlene Modus. Der alte rohe TCP-Modus (RAW) braucht eigene Portfreigaben und wird nur noch in Altinstallationen vorgehalten.

Repositories und State in NiFi 2.x

- **flowfile_repository/** — Metadaten aller aktiven FlowFiles (Write-ahead-Log). Klein, aber kritisch.
- **content_repository/** — die eigentlichen FlowFile-Inhalte (Content Claims). Wächst mit Volumen.
- **provenance_repository/** — Event-Log für Data Provenance. **Wächst schnell** — Retention bewusst setzen.
- **state/** — Knoten- und Cluster-State. Im Cluster über ZooKeeper koordiniert.

Empfehlung produktiv: jedes Repo auf eine **eigene Disk/Partition** legen. Diskspace-Alarm rechtzeitig konfigurieren.

Typische Tuning-Fehler

- Parallelität hochdrehen, ohne Zielsysteme oder Fehlerraten zu beobachten
- Backpressure-Grenzen zufällig setzen
- Queue-Wachstum nicht als Frühwarnsignal interpretieren
- Cluster einführen, obwohl der eigentliche Engpass im Design oder in Fremdsystemen liegt
- Test-, Last- und Recovery-Szenarien nicht vorab simulieren

| Modul 6 — Betrieb, Monitoring, Live-Quality, Migration

Was im Alltag wirklich anliegt: Logs lesen, Lasten beobachten, Daten validieren, Flows umziehen.

Logging in NiFi — die wichtigen Logfiles

NiFi schreibt nach `logs/` unter dem Installationsverzeichnis. Drei Dateien lohnen den Blick:

- `nifi-app.log` — zentraler Anwendungslog: Prozessor-Exceptions, Stacktraces, Lifecycle.
- `nifi-bootstrap.log` — Start-/Stopp-Vorgänge, JVM-Argumente, Bootstrap-Probleme.
- `nifi-user.log` — Authentifizierung, Autorisierung, Policy-Verstöße.

Optional aktivierbar: `nifi-request.log` (HTTP-API-Zugriffe) und `nifi-deprecation.log`.

Loglevel und Log-Konfiguration

- konfiguriert in `conf/logback.xml`
- pro Klasse / pro Logger einstellbar — z. B. einen einzelnen Prozessor temporär auf `DEBUG`
- **Live-Tipp:** Logback erkennt Änderungen ohne Neustart (Scan-Intervall in `logback.xml`)
- Rolling-File-Policy mit `maxHistory` und `totalSizeCap` aktiv halten

 **Falle:** `DEBUG` auf `org.apache.nifi.processors.standard` lässt das `nifi-app.log` in Minuten explodieren. Gezielt scopen.

Live-Monitoring der Jobs

1. **Canvas-Metriken** — pro Prozessor: In/Out, Read/Write Bytes, Tasks, Time. 5-Minuten-Rolling-Fenster.
2. **Status History** — Rechtsklick → *View status history*: Trend pro Metrik über Stunden/Tage.
3. **Summary / Cluster Overview** — übergreifender Blick über alle Prozessoren und Knoten.

Für Alarme nach außen: **Reporting Tasks** unter *Controller Settings* → *Reporting Tasks* — z. B. `SiteToSiteStatusReportingTask` oder `MonitorMemory`.

MonitorActivity und Reporting Tasks

- **MonitorActivity** — Prozessor im Flow selbst. Erzeugt ein FlowFile, wenn ein erwarteter Datenfluss **ausbleibt**.
- nachgelagert: **PutEmail**, **InvokeHTTP** (Webhook), oder eine eigene Alarm-Group
- **Reporting Tasks** laufen außerhalb der Flows — pushen Metriken in Prometheus, AMS, S2S-Endpoints
- Kombination: **MonitorActivity** für **fachliche** Lücken, Reporting Tasks für **technische** Metriken

Praxispunkt: „Datenfluss steht still“ ist meist kein Crash, sondern eine **stille Quelle**. **MonitorActivity** ist der einzige Weg, das direkt im Flow zu erkennen.

Externe Alarmierung — Slack, PagerDuty, E-Mail

Reporting Tasks pushen Metriken nach außen — für **Alarme an einen Menschen** braucht es einen Kanal. Drei eingebaute Wege, alle ohne Custom-Code:

Kanal	Prozessor / Mechanik
Slack	PutSlack direkt, oder InvokeHTTP gegen einen Slack-Incoming-Webhook
PagerDuty / Opsgenie	InvokeHTTP gegen die Events-API
E-Mail	PutEmail (SMTP-Properties) — Klassiker für nicht-zeitkritische Hinweise
Webhook generisch	InvokeHTTP POST mit application/json — Microsoft Teams, eigene ChatOps-Bots

Typisches Muster:



⚠ Webhook-URLs und Routing-Keys gehören in **Sensitive Parameters** — sonst sind die Alarm-Kanäle nach dem ersten Git-Commit kompromittiert.

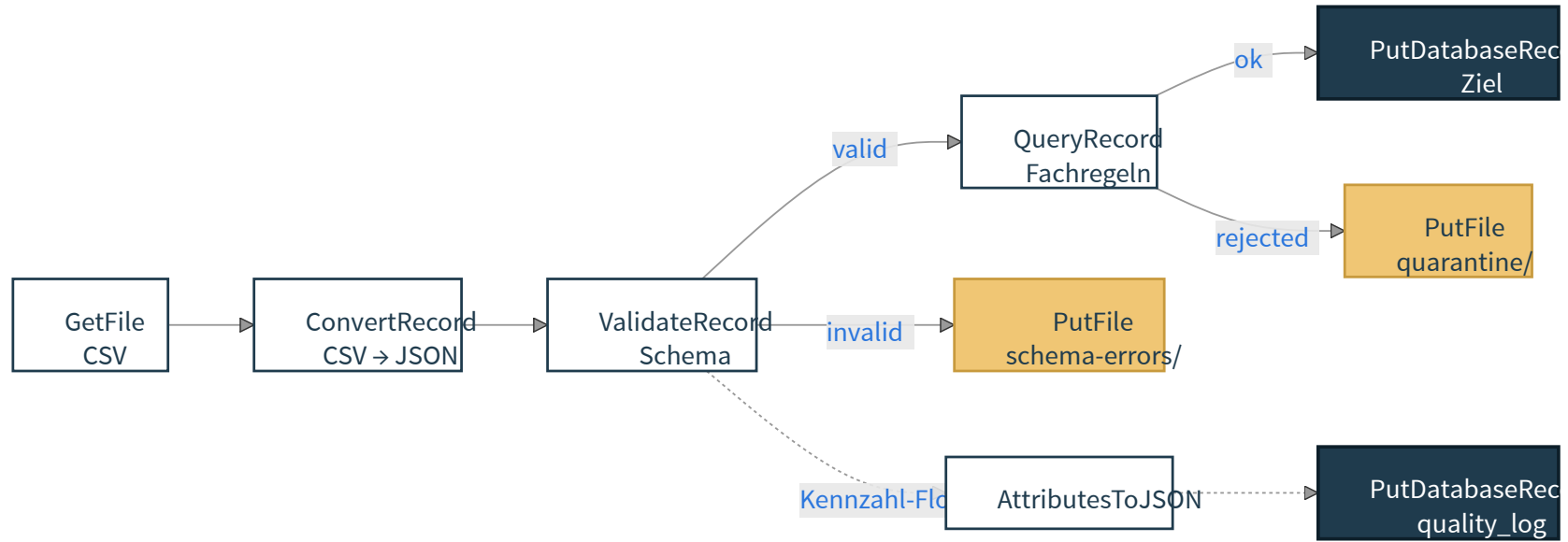
Live-Quality der Datenflüsse

Datenqualität wird als **dedizierter Pfad** modelliert, nicht als Beifang:

- **ValidateRecord** prüft jedes Record-FlowFile gegen ein Schema — Routen: `valid`, `invalid`, `failure`
- **RouteOnAttribute** klassifiziert anhand fachlicher Regeln (z. B. `${amount:gt(0)}`)
- **Quarantäne-Pfad** legt invalide Datensätze separat ab — niemals stillschweigend verwerfen
- **Kennzahl-Flow**: zusätzlich ein paralleler Pfad mit `AttributesToJSON` + `PutDatabaseRecord` in eine Quality-Tabelle

Faustregel: Jeder Produktionsflow hat mindestens drei Outputs — *clean*, *quarantine*, *quality-metrics*.

Demo — Quality-Flow konkret



Migration: Use Cases nach Serverumzug

Drei realistische Szenarien — je nach Reifegrad:

- **Mit Git-basiertem Flow Registry Client** (empfohlen) — Flows liegen versioniert im Git. Neuer Server zeigt auf dasselbe Repository und zieht die aktuelle Version.
- **Ohne Versionierung, mit Flow Definition** — Process Group rechtsklick → *Download flow definition* exportiert ein JSON. Auf dem neuen Server hochladen.
- **Notnagel** — Kopie von `conf/flow.json.gz` auf Zielmaschine. Funktioniert, hinterlässt aber keine Historie.

Was immer mitwandern muss: `flow.json.gz`, `conf/authorizers.xml` / `users.xml`,
Keystores/Truststores, Parameter-Werte (Secrets!).

Migration: Was beim Umzug ausreißt

- **Controller Services mit Verbindungsdaten** — Passwörter sind mit dem `nifi.sensitive.props.key` der alten Instanz verschlüsselt
- **Lokale Pfade** in Prozessoren (`/data/in/...`) — auf neuem Server prüfen oder über Parameter Contexts abstrahieren
- **Cluster-spezifische State-Daten** — `state-management.xml` und ZooKeeper-Konfiguration
- **Provenance-Repository** — wandert in der Regel **nicht** mit; alte Historie bleibt auf altem Server

Reihenfolge: Testumgebung zuerst → Smoketest mit einem Flow → erst dann produktiv migrieren.

Migration: Talend Jobs nach NiFi

Konzeptuelle Übersetzung — kein automatischer Konverter:

Talend	NiFi-Äquivalent
tFileInputDelimited / tFileOutputDelimited	GetFile + ConvertRecord (CSVReader/Writer)
tMap (Mapping/Lookup)	UpdateAttribute + JoltTransformJSON oder QueryRecord
tFilterRow	RouteOnAttribute oder QueryRecord (WHERE)
tDBInput / tDBOutput	QueryDatabaseTable / PutDatabaseRecord
tRunJob (Subjob-Aufruf)	Process Group als Unterstruktur
tPreJob / tPostJob	nicht 1:1 — über Wait / Notify oder steuernde Process Group

Talend → NiFi

- Talend denkt **batch und schritt-orientiert** — NiFi denkt **fluss-, daten- und zustandsgetrieben**
- ein Talend-Job mit 30 Komponenten wird **nicht** zu einer Process Group mit 30 Prozessoren
- **Vorgehen empfehlenswert:** zuerst Datenfluss skizzieren (Quelle, Transformation, Ziel, Fehler), dann pro Schritt das passende NiFi-Konstrukt wählen
- Lookups gegen Stammdaten: in NiFi oft `LookupRecord` + Map-Cache-Controller-Service statt eines `tMap` -Bocks

Realistisch: Migration ist 30 % Werkzeug, 70 % **Re-Design**.

Connectoren zu SAP und P&I

NiFi hat keinen offiziellen SAP-Connector.

- **JDBC zu SAP HANA** — `QueryDatabaseTable` / `ExecuteSQL` mit HANA-JDBC-Treiber als externes JAR
- **OData / SAP Gateway** — `InvokeHTTP` gegen `/sap/opu/odata/...`, Authentifizierung via Header oder OAuth-Controller-Service
- **Dateischnittstellen** — `GetSFTP` / `ListSFTP` auf SAP-Export-Verzeichnisse, dann `ConvertRecord` aus IDoc/CSV
- **Kommerziell** — Connector-Pakete von Hortonworks/Cloudera-Partnern oder eigene NAR-Builds

P&I: ähnlich pragmatisch — meist über REST/Datei. Vorab klären, welche Schnittstelle das System bereitstellt.

SAP/P&I — Praxis-Hinweise für den Trainer

- **Treiber-JARs** kommen nach `lib/` oder besser als eigene **NAR** (NiFi Archive) verpackt
- **Credentials niemals** als Klartext im Prozessor — Sensitive Parameter mit Provider
- **OData**: Pagination (`$skip` / `$top` oder `next.url`) muss im Flow modelliert werden — typisch über Attribute wie `page` , `skip` , `next.url` plus `InvokeHTTP` / `RouteOnAttribute` -Schleife
- **IDoc / RFC**: nicht direkt; entweder über Middleware (PI/PO, Cloud Integration) abholen oder über kommerzielle Connectoren

Ehrliche Aussage im Kurs: SAP-Integration mit NiFi ist machbar, aber kein Plug-and-Play. Erwartungsmanagement gehört in die ersten 10 Minuten der Diskussion.

SAP/P&I — Entscheidungsfragen vor dem Flow-Design

Klären, **bevor** der erste Prozessor aufs Canvas wandert:

- **Welche Schnittstelle stellt das Quell-/Zielsystem wirklich bereit?** REST, OData, JDBC, RFC, BAPI, File-Drop, IDoc?
- **Wer betreibt Credentials und Zertifikate?** SAP-Basis-Team, NiFi-Admin, externe Middleware?
- **Pagination, Rate Limits, Transaktionen, Idempotenz?** — bestimmt Retry- und Quarantäne-Logik
- **Delta-Logik:** wie wird Wiederlesen vermieden — Zeitstempel, Change-Pointer, CDC?
- **Was passiert bei Fehler im Zielsystem?** — Rollback, Kompensation, Replay?

Modul 7 — Deployment, Security, Multi-User

Von der Entwicklungsinstanz zum stabilen Betriebsmodell.

Dev, Test und Produktion sauber trennen

- **Entwicklung** dient dem Modellieren, Probieren und schnellen Iterieren
- **Test / Staging** überprüft Integrationsverhalten, Last, Fehlerpfade und Berechtigungen
- **Produktion** priorisiert Stabilität, Nachvollziehbarkeit, Security und kontrollierte Änderungen
- Parameter Contexts und Git-Versionierung erleichtern die Trennung ohne Flow-Kopienwildwuchs
- zusätzliche Pflichtlektüre: **Administration Guide, TLS/SSL und Authorizations**

Betrieb, Sicherheit und Administration

- produktive NiFi-Umgebungen brauchen klare Regeln für **TLS/SSL, Benutzerrechte und Autorisierung**
- Logging, Zugriffsschutz und sichere Controller Services gehören zur Betriebsarchitektur
- Cluster-Betrieb erhöht die Anforderungen an Zertifikate, Node-Kommunikation und Governance
- Security-Fragen sollten im Training nicht erst am Ende als Randthema auftauchen

Ein Flow ist erst dann produktionsreif, wenn er nicht nur funktioniert, sondern auch **sicher freigegeben und betrieben** werden kann.

Mehrbenutzer in NiFi — die zwei Bausteine

NiFi trennt **Identität** und **Autorisierung**:

- **User/Group Provider** — wer existiert. Datei (`FileUserGroupProvider`), LDAP (`LdapUserGroupProvider`) oder Azure Graph
- **Access Policy Provider** — wer darf was. Auf Ressourcen wie `/flow` , `/restricted-components` , einzelne Process Groups
- konfiguriert in `conf/authorizers.xml` — Vorlagen für File-basiert und LDAP sind enthalten
- der **Initial Admin Identity** ist der erste User, der überhaupt Policies vergeben darf — wird genau einmal gesetzt

Demo vs. Produktion: Der Single-User-Modus nutzt generierte Credentials und ist für lokale Übungen praktisch. Produktiver Mehrbenutzerbetrieb nutzt **LDAP/OIDC/TLS-Identitäten** plus Access Policies.

Access Policies konkret

Policies sind paarweise: (**Ressource, Aktion**). Wichtige Ressourcen:

- `/flow` — Canvas sehen
- `/controller` — Controller Services / Reporting Tasks verwalten
- `/policies` — Policies selbst ändern (die Meta-Berechtigung)
- `/restricted-components` — Prozessoren mit Dateisystemzugriff
- einzelne Process Groups — sehen, ändern, starten, Provenance lesen

Vererbung: Policies an einer Process Group **vererben sich auf Unterkomponenten**, bis explizit überschrieben.

Mehrbenutzer — typische Rollen-Schnitte

- **Flow-Entwickler** — Lese/Schreib auf eigene Process Group, kein Zugriff auf `/controller` oder `/policies`
- **Flow-Operator** — Start/Stop und Provenance auf produktive Groups, kein Editrecht
- **NiFi-Admin** — Vollzugriff inkl. `/policies` und `/controller`, üblicherweise zwei Personen
- **Auditor / Read-only** — `/flow` und Provenance global, sonst nichts

Praxis: Mit `LdapUserGroupProvider` lassen sich diese Rollen auf bestehende AD-Gruppen abbilden — Policies referenzieren dann Gruppen, nicht einzelne Nutzer.

SSL/TLS für Webclient-Verbindung

NiFi erzwingt HTTPS, sobald TLS konfiguriert ist. Gebraucht werden zwei Stores:

- **Keystore** (`keystore.jks` / `.p12`) — privates Server-Zertifikat, das NiFi vorzeigt
- **Truststore** (`truststore.jks` / `.p12`) — Zertifikate, denen NiFi vertraut
- gepflegt in `conf/nifi.properties`: `nifi.security.keystore*` und `nifi.security.truststore*`
- `nifi.web.https.host` + `nifi.web.https.port` setzen — `nifi.web.http.port` leeren

SSL/TLS — Standardweg in NiFi 2.x

- **Standard:** Zertifikate aus der **Unternehmens-CA** oder via **OpenSSL** + bestehender PKI
- den **Subject DN** kennen — daraus entsteht die NiFi-Identität für Access Policies
- **SAN** (Subject Alternative Name) muss den **Hostnamen** enthalten — sonst Mismatch
- nach Zertifikatswechsel: NiFi-Service-Neustart, **nicht** nur Browser-Refresh
- für Cluster: alle Knoten brauchen gegenseitig Vertrauen — gleiche CA in jedem Truststore
- **Single-Node-Dev:** NiFi 2.x richtet beim ersten Start automatisch ein self-signed-Set ein — Demo, nicht Produktion

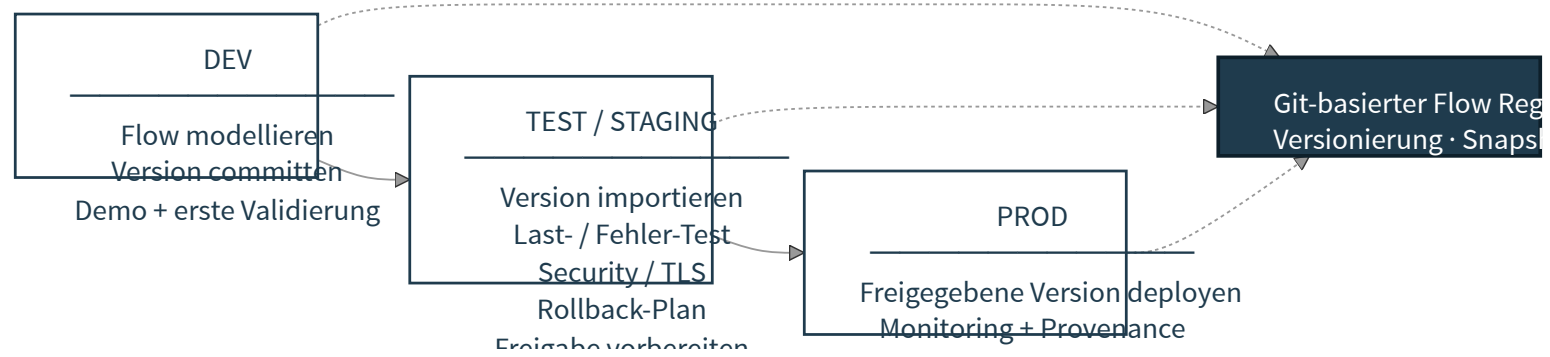
StandardSSLContextService — TLS *im* Flow

Ein **Controller Service** für TLS-Verbindungen aus dem Flow heraus — nicht zu verwechseln mit der Web-UI-TLS:

- referenziert von `InvokeHTTP`, `ListenHTTP`, `ConsumeKafka`, `PutKafkaRecord`, `PostHTTP`, ...
- bündelt Keystore + Truststore + TLS-Protokolle (`TLSv1.2`, `TLSv1.3`)
- **ein** Service kann von vielen Prozessoren genutzt werden — zentrale Stelle bei Zertifikatsrotation
- empfohlen: einer pro **Vertrauensdomäne**, nicht einer pro Prozessor

Warum getrennt: Web-UI-TLS schützt den Zugriff *auf* NiFi. `StandardSSLContextService` schützt Verbindungen, die NiFi *aufbaut*.

Standalone vs. Cluster-Deployment



Git-Versionierung, Snapshots und Rollback

- Versionierte Flows schaffen reproduzierbare Stände
- **Snapshots** helfen bei Releases, Audits und Fehleranalysen
- Rollback muss nicht improvisiert, sondern als **normaler Betriebsfall** gedacht werden
- gemeinsam mit Git-basierter Versionierung entsteht ein belastbarer Änderungs- und Freigabeprozess

Best Practices für professionelle NiFi-Umgebungen

- klare Namenskonventionen für Prozessoren, Verbindungen und Attribute
- Beziehungen nie unverdrahtet lassen
- Parameter und Controller Services statt Hardcoding
- Fehlerpfade, Retry-Strategien und Dead-Letter-Muster früh einplanen
- Provenance, Monitoring und Doku **von Anfang an** einplanen
- Performance erst messen, dann tunen